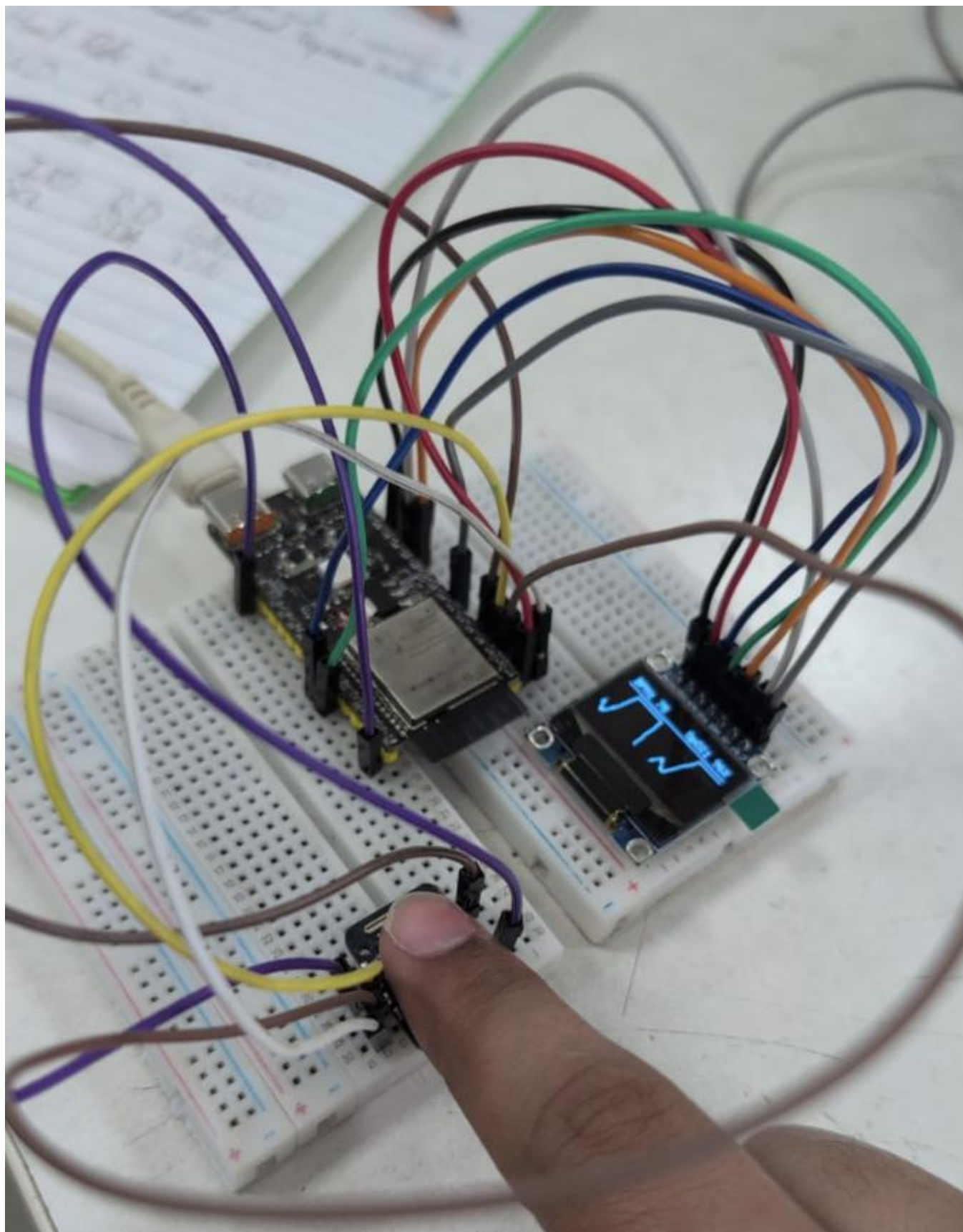




Smart Pulse Oximeter Playbook

I. Challenge

Develop an embedded biometric acquisition terminal using Photoplethysmography (PPG) that isolates high-frequency micro-volumetric blood changes. The system must cleanly separate unshared high-speed hardware bus lines, perform dynamic DC-bias filtering, compute heart rate (BPM) and blood oxygen saturation (SpO_2) over deterministic windowing blocks, and stream real-time physiological waveforms to both an SPI-driven OLED display and diagnostic serial plotters.

II. Guide

A. Concept

i. Underlying Engineering Principles & Reference Links

1. **Differential Light Absorption in Hemoglobin:** Arterial blood volume pulses during cardiac cycles. Deoxygenated Hemoglobin (Hb) absorbs more Red light (660 nm), whereas Oxygenated Hemoglobin (HbO_2) absorbs more Infrared light (940 nm).

- **Learn More:** [NCBI – Principles of Pulse Oximetry](#)

2. **Ratio of Ratios (SpO_2) Calculation:** The optical signal is split into a large static component (DC) from resting tissue and a dynamic component (AC) from pulsating arterial blood. The normalized absorption ratio (R) maps directly to systemic oxygenation:

$$R = \frac{\left(\frac{\text{AC}_{\text{Red}}}{\text{DC}_{\text{Red}}}\right)}{\left(\frac{\text{AC}_{\text{IR}}}{\text{DC}_{\text{IR}}}\right)}, \quad \text{SpO}_2 = 104 - 17 \cdot R$$

- **Learn More:** [Pulse Oximetry Understanding](#)

3. **Exponential Moving Average (EMA) DC Filtering:** Raw photodiode data rides on a massive baseline voltage. To extract the micro-AC waveform for graph rendering, an exponential filter continuously tracks and subtracts the rolling DC component:

$$\text{DC}_t = (\beta \cdot \text{DC}_{t-1}) + ((1 - \beta) \cdot \text{IR}_{\text{Sample}}), \quad \text{AC}_{\text{Display}} = \text{IR}_{\text{Sample}} - \text{DC}_t$$

- **Learn More:** [DSP Related – Digital Filters for DC Offset Removal](#)

4. **Deterministic Block Processing Window:** Evaluating metrics sample-by-sample introduces phase jitter, micro-stutters, and computational drift. Collecting 100-sample array buffers (1.0-second windows at 100 Hz) enables global vector calculations without stalling real-time signal acquisition.

- **Learn More:** [Block-Based Signal Processing](#)

5. **Bus Isolation Across Mixed Peripherals:** To prevent data corruption on high-speed hardware lines, write-only SPI display lines and bidirectional I²C sensor buses share system power domains while maintaining strictly isolated controller pins.

- **Learn More:** [Communication Protocols](#)

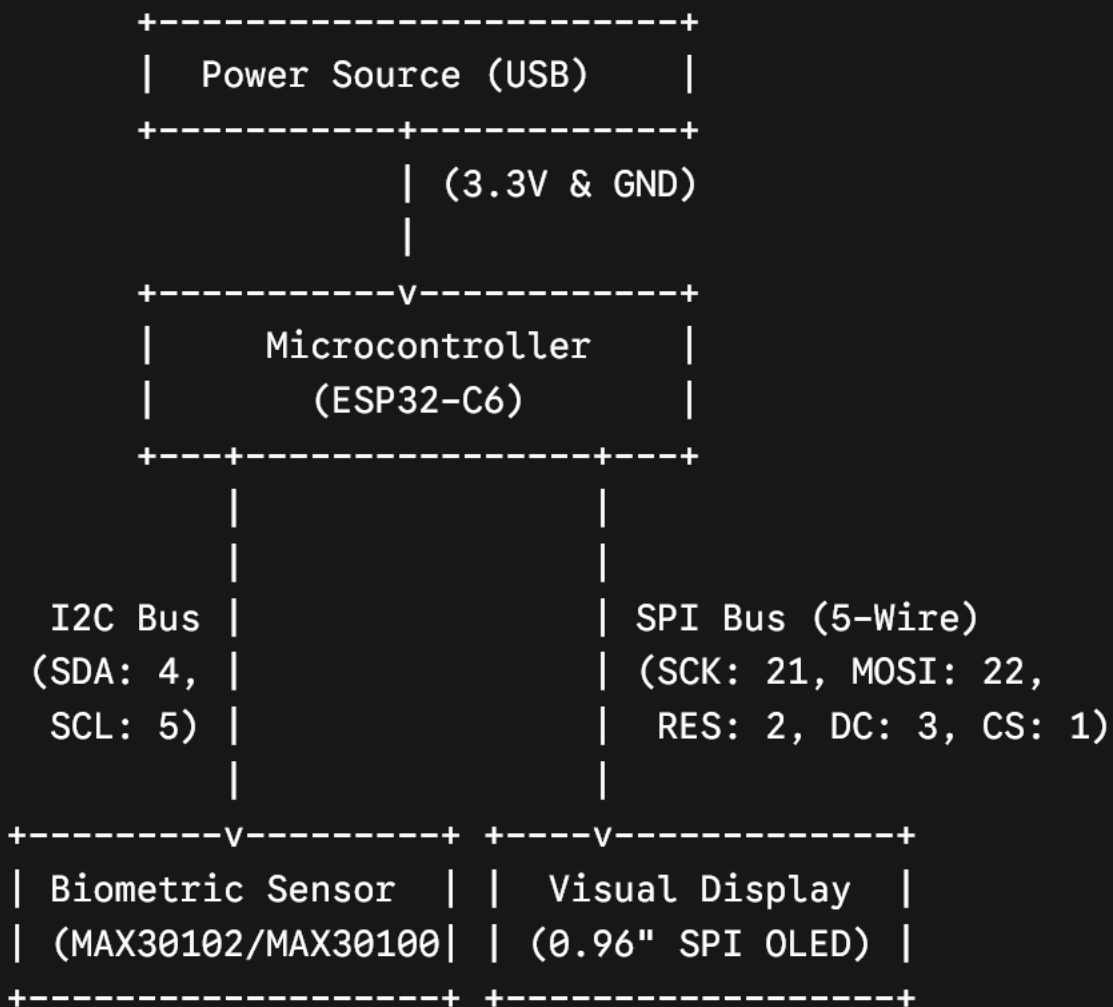
ii. Real-World Example

Medical-grade ICU vital sign monitors, commercial fitness wearables (such as Apple Watch, Garmin, or Fitbit), and emergency pulse oximeter clip units. These systems use optical LEDs and photodiodes to continuously monitor heart rate variability and blood oxygen saturation in real time.

B. Components

- **Microcontroller:** WeAct Studio ESP32-C6 (32-bit RISC-V SoC)
- **Biometric Sensor:** MAX30102 High-Sensitivity Pulse Oximeter & Heart-Rate Breakout
- **Display Peripheral:** 0.96-inch SPI OLED Display (SSD1306 driver)
- **Interconnect Base:** Solderless breadboard and jumper wires

Block Diagram



C. Connections

Sensor/Display Pin	ESP32-C6 Pin	Signal Type	Functional Description
OLED GND / MAX30102 GND	GND	Power	System Common Ground Plane
OLED VDD / MAX30102 VIN	3.3V or 5V	Power	3.3V Logic / 5V LED Headroom Rail
OLED SCK	GPIO 21	Output (FSPI)	Hardware SPI Clock Master Line
OLED SDA	GPIO 22	Output (FSPI)	Hardware SPI Data Line (MOSI)
OLED RES	GPIO 2	Output	Discrete Hardware Reset Flag
OLED DC	GPIO 3	Output	Data / Command Selection (0 = Cmd, 1 = Data)
OLED CS	GPIO 1	Output (FSPI)	Active-Low Peripheral Chip Select
MAX30102 SCL	GPIO 5	In/Out (I ² C)	Hardware I ² C Clock Master Line (400 kHz)
MAX30102 SDA	GPIO 4	In/Out (I ² C)	Hardware I ² C Serial Data Line

Hardware Note:

- The IRD and RD pins broken out on this MAX30102 variant are left floating. They represent low-side open cathodes of internal LEDs and are managed automatically via internal current-sink transistors in software.
- You will have to install the “Sparkfun MAX30102/MAX30105” library through the Arduino IDE’s Library Manager

D. Code

```
1. C++
2. #include <Arduino.h>
3. #include <Wire.h>
4. #include <SPI.h>
5. #include <U8g2lib.h>
6. #include "MAX30105.h"
7.
8. #define SPI_OLED_SCK 21
9. #define SPI_OLED_SDA 22
10. #define SPI_OLED_RES 2
11. #define SPI_OLED_DC 3
12. #define SPI_OLED_CS 1
13.
14. #define I2C_SDA 4
15. #define I2C_SCL 5
16.
17. U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI oled(U8G2_R0, SPI_OLED_CS, SPI_OLED_DC, SPI_OLED_RES);
18. MAX30105 particleSensor;
19.
20. const int SCREEN_WIDTH = 128;
21. int waveBuffer[SCREEN_WIDTH];
22.
23. const int BUFFER_SIZE = 100;
24. uint32_t redBuffer[BUFFER_SIZE];
25. uint32_t irBuffer[BUFFER_SIZE];
26. int bufferIndex = 0;
27.
28. float dcIR = 0;
29. const float beta = 0.95f;
30.
31. float finalBpm = 0.0f;
32. float finalSpO2 = 0.0f;
33. bool fingerPresent = false;
34.
35. void setup() {
36.     Serial.begin(115200);
37.
38.     Wire.begin(I2C_SDA, I2C_SCL, 400000U);
39.     SPI.begin(SPI_OLED_SCK, -1, SPI_OLED_SDA, SPI_OLED_CS);
40.     oled.begin();
41.
42.     if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
43.         while (1); // Trap configuration lockup if I2C fails
44.     }
45.
46.     // 100Hz dual-mode initialization
47.     particleSensor.setup(50, 1, 2, 100, 411, 4096);
48.
49.     for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
50. }
51.
52. void loop() {
53.     while (particleSensor.check() > 0) {
54.         uint32_t redSample = particleSensor.getFIFORed();
55.         uint32_t irSample = particleSensor.getFIFOIR();
56.
57.         // 1. Hardware Presence Gate
58.         if (irSample < 25000) {
59.             fingerPresent = false;
```

```

60.     finalBpm = 0.0f;
61.     finalSpO2 = 0.0f;
62.     bufferIndex = 0;
63.     for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
64.     particleSensor.nextSample();
65.     break;
66. }
67. fingerPresent = true;
68.
69. // 2. Continuous Visual Waveform Filter for OLED Graph
70. dcIR = (beta * dcIR) + ((1.0f - beta) * irSample);
71. float acDisplay = irSample - dcIR;
72.
73. for (int i = 0; i < SCREEN_WIDTH - 1; i++) {
74.     waveBuffer[i] = waveBuffer[i + 1];
75. }
76. int yPixel = 42 - (int)(acDisplay * 0.05f);
77. waveBuffer[SCREEN_WIDTH - 1] = constrain(yPixel, 18, 54);
78.
79. // 3. Populate Array Block
80. redBuffer[bufferIndex] = redSample;
81. irBuffer[bufferIndex] = irSample;
82. bufferIndex++;
83.
84. // 4. Vector Analytics Window Execution (Once per 100 samples)
85. if (bufferIndex >= BUFFER_SIZE) {
86.     uint32_t minRed = redBuffer[0], maxRed = redBuffer[0];
87.     uint32_t minIR = irBuffer[0], maxIR = irBuffer[0];
88.     uint64_t sumRed = 0, sumIR = 0;
89.
90.     for (int i = 0; i < BUFFER_SIZE; i++) {
91.         if (redBuffer[i] < minRed) minRed = redBuffer[i];
92.         if (redBuffer[i] > maxRed) maxRed = redBuffer[i];
93.         if (irBuffer[i] < minIR) minIR = irBuffer[i];
94.         if (irBuffer[i] > maxIR) maxIR = irBuffer[i];
95.         sumRed += redBuffer[i];
96.         sumIR += irBuffer[i];
97.     }
98.
99.     float dcR_calc = (float)sumRed / BUFFER_SIZE;
100.    float dcI_calc = (float)sumIR / BUFFER_SIZE;
101.    float acR_calc = (float)(maxRed - minRed);
102.    float acI_calc = (float)(maxIR - minIR);
103.
104.    // 5. SpO2 Computational Node
105.    if (acI_calc > 20 && acR_calc > 20) {
106.        float R = (acR_calc / dcR_calc) / (acI_calc / dcI_calc);
107.        float spo2Est = 104.0f - (17.0f * R);
108.        if (spo2Est >= 90.0f && spo2Est <= 100.0f) {
109.            finalSpO2 = spo2Est;
110.        } else if (spo2Est < 90.0f && spo2Est >= 75.0f) {
111.            finalSpO2 = (0.5f * finalSpO2) + (0.5f * spo2Est);
112.        }
113.    }
114.
115.    // 6. Midpoint Inflection Pitch Timing (BPM calculation)
116.    int lastPeakIdx = -1;
117.    int totalIntervals = 0;
118.    int intervalSum = 0;
119.    float localThresh = dcI_calc;
120.

```



```

121.     for (int i = 1; i < BUFFER_SIZE - 1; i++) {
122.         if (irBuffer[i] > localThresh && irBuffer[i-1] <= localThresh) {
123.             if (lastPeakIdx != -1) {
124.                 intervalSum += (i - lastPeakIdx);
125.                 totalIntervals++;
126.             }
127.             lastPeakIdx = i;
128.         }
129.     }
130.
131.     if (totalIntervals > 0) {
132.         float avgIntervalSamples = (float)intervalSum / totalIntervals;
133.         float calculatedBpm = (60.0f * 100.0f) / avgIntervalSamples;
134.         if (calculatedBpm >= 50.0f && calculatedBpm <= 160.0f) {
135.             finalBpm = calculatedBpm;
136.         }
137.     }
138.     bufferIndex = 0;
139. }
140. particleSensor.nextSample();
141. }
142.
143. // 7. Render & Screen UI Generation Path
144. oled.clearBuffer();
145.
146. if (fingerPresent && finalBpm > 10.0f) {
147.     for (int x = 1; x < SCREEN_WIDTH; x++) {
148.         oled.drawLine(x - 1, waveBuffer[x - 1], x, waveBuffer[x]);
149.     }
150.     oled.setFont(u8g2_font_6x10_tf);
151.     oled.setCursor(2, 11); oled.print("BPM: "); oled.print((int)finalBpm);
152.     oled.setCursor(72, 11); oled.print("SpO2: "); oled.print((int)finalSpO2);
oled.print("%");
153. } else {
154.     // Standardized typography layout to guarantee 128px bounding limits
155.     oled.setFont(u8g2_font_5x7_tf);
156.     oled.setCursor(34, 32); oled.print("PLACE FINGER");
157.     oled.setCursor(36, 44); oled.print("ON THE LENS");
158. }
159.
160. oled.drawLine(0, 14, 128);
161. oled.drawLine(0, 56, 128);
162. oled.sendBuffer();
163. }

```

Student Tip: Open the Serial Monitor (Ctrl+Shift+M) at 115200 baud to monitor real-time raw values and vector computations.

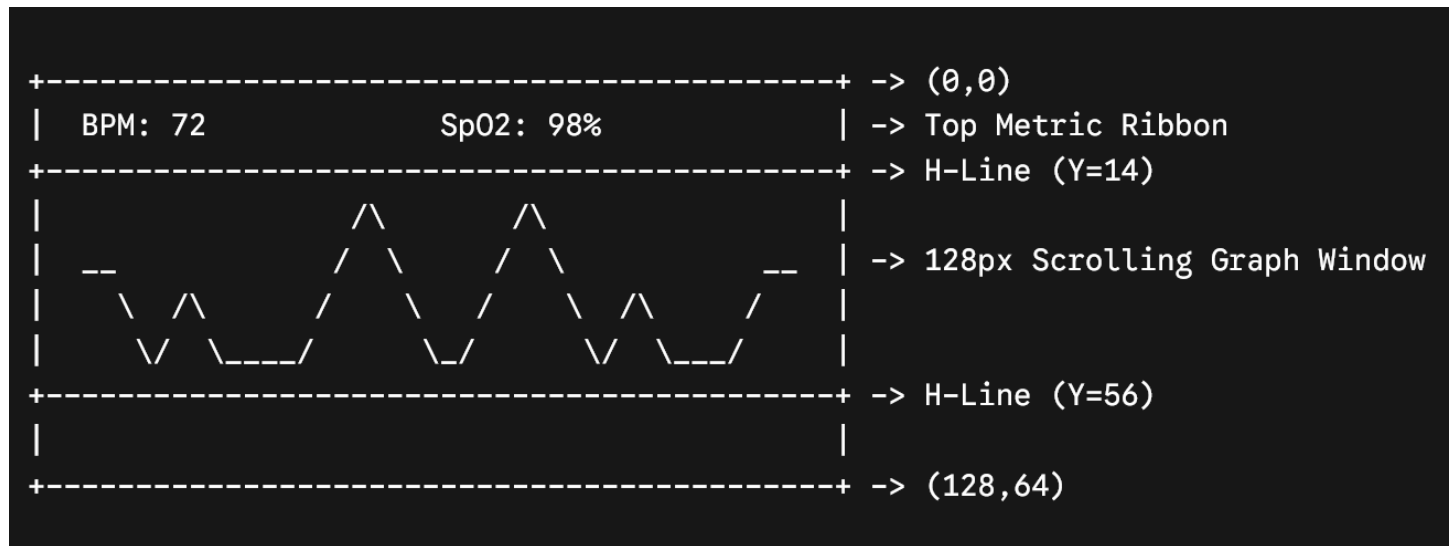
To visually evaluate signal quality and watch the real-time arterial pulse wave, switch to the Serial Plotter (Ctrl+Shift+L). The output telemetry formats IR_Raw, Filtered_AC, BPM, and SpO2 into synchronized multi-colored curves. Watching Filtered_AC helps demonstrate how motion artifacts or excessive contact pressure flatten the cardiac pulse wave.

Operational Guidelines & UI Validation

1. Interface Layout Geometry

The UI splits the 64-pixel vertical space into three strict windows:

- Top Ribbon (0 to 14px): Displays digital values for BPM and percentage bounds.
- Graph Canvas (15 to 55px): Displays the continuous filtered AC waveform.
- Bottom Bar (56 to 64px): Visual alignment limit line.



2. Diagnostic & Use Checklist

1. **Zero-Power Auto-Reset:** Verify that when a finger is completely removed, the raw IR signal falls below 25,000. The interface must instantly transition from displaying metric values to rendering a centered "PLACE FINGER / ON THE LENS" layout using the optimized 5-pixel width typography matrix. The system may take 15 seconds for it to co
2. **Contact Pressure:** Place the finger pad lightly over the sensor. Excess force will compress the arterial capillary bed, which zero-flattens the AC pulse and prevents the array from registering local midpoint inflections.

Use this section in case of parts variation

Hello! This section is designed to assist in the case of components variance. In simpler terms, did you get a component that has the same functions but the connections are different?

Some examples include:

- Your OLED Display has 4 pins instead of 7 pins? It turns out you got an [I2C Version!](#)
- Your buzzer module is either 2 pins or 3 pins
- Your sensors are a different variation

So here is a quick comparison regarding the Pulse Oximeter Sensor. Observe the central black IC on the respective PCBs. Also note that the arduino libraries used for these sensors will be different but the main code and wiring connections are unchanged.

MAX30100	MAX30102
 A green PCB module for the MAX30100 sensor. It features a central black IC, two yellow capacitors, and several resistors. The PCB is labeled 'MAX30100/30102' and 'VIN SCL SDA INT IRD RO GND'.	 A green PCB module for the MAX30102 sensor. It features a central black IC, two yellow capacitors, and several resistors. The PCB is labeled 'MAX30102' and 'VIN SCL SDA INT IRD RO GND'.
Use the “MAX30100lib” Library from the Arduino IDE’s library manager	Use the “Sparkfun MAX30105” Library from the Arduino IDE’s library manager

Technical Specification Comparison

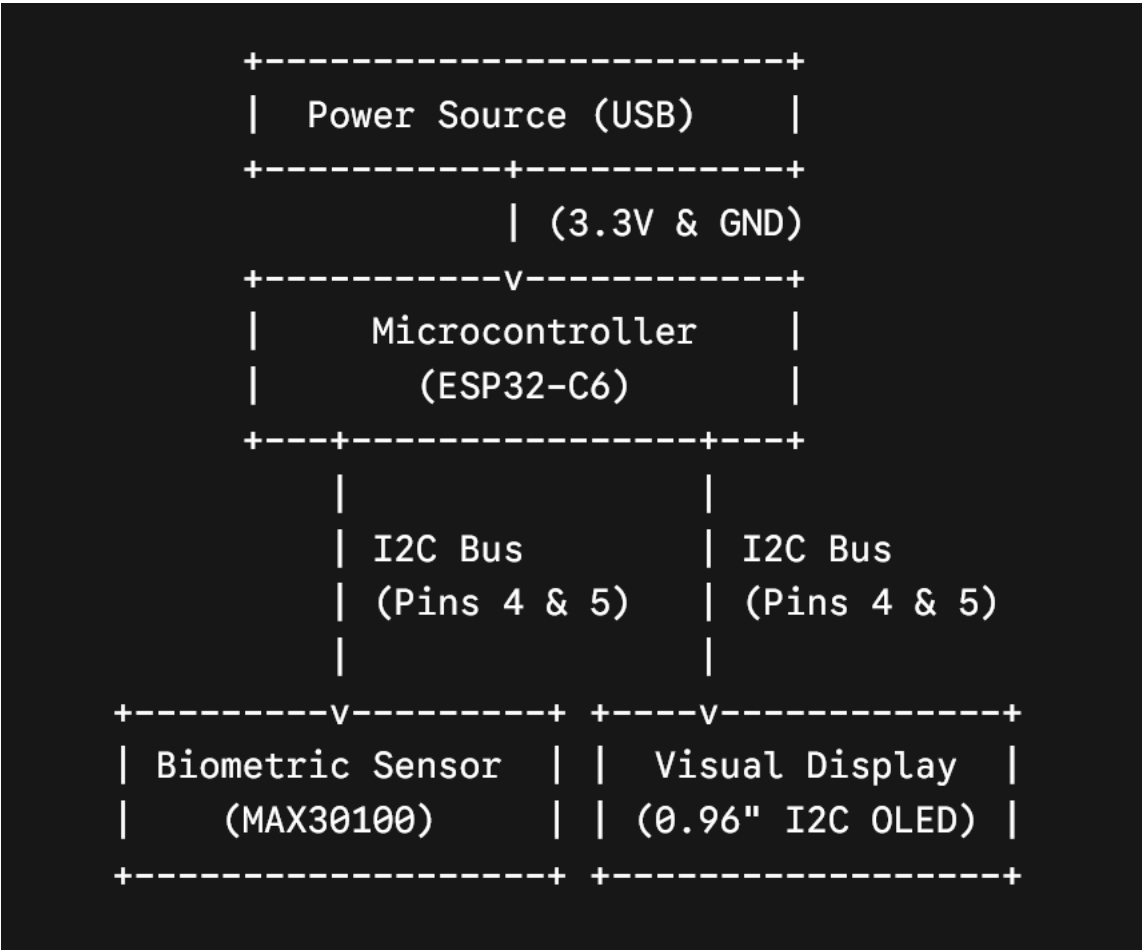
While both sensors operate on photoplethysmography (PPG) using Red and Infrared (IR) LEDs, their internal hardware architectures differ substantially:

Feature	MAX30100 Module	MAX30102 Module
Status	Discontinued / Legacy	Active / Recommended
ADC Resolution	16-bit Sigma Delta	18-bit Sigma Delta (Higher sensitivity)
FIFO Buffer	16 samples	32 samples (Fewer CPU interruptions)

Power Consumption	Higher average current draw	Lower current draw (Narrower programmable LED pulse widths)
Additional Features	Standard ambient cancellation	Proximity detection, improved ambient light cancellation
Communication	I2C (Up to 400kHz)	I2C (Up to 400kHz)

For the Smart Pulse Oximeter, we are now focusing on how to use the **I2C OLED display** and the older **MAX30100 heart rate sensor**. The wiring should be noticeably simpler to work with.

Block Diagram:



You will need to download the “**MAX30100lib**” library in your Arduino IDE’s Library Manager

Diagnostic & Use Checklist

1. **Zero-Power Auto-Reset:** Verify that when a finger is completely removed, the raw IR signal falls below 25,000. The interface must instantly transition from displaying metric values to rendering a centered "PLACE FINGER / ON THE LENS" layout using the optimized 5-pixel width typography matrix. The system may take 15 seconds to clear it’s buffer and be ready to activate once more.

2. **Contact Pressure:** Place the finger pad lightly over the sensor. Excess force will compress the arterial capillary bed, which zero-flattens the AC pulse and prevents the array from registering local midpoint inflections.

Connections (I2C Version)

Component Pin	ESP32-C6 Pin	Description
Sensor GND & OLED GND	GND	Common ground connection (Minus).
Sensor VIN & OLED VCC	3.3V	Provides logic power to turn the modules on.
Sensor SDA & OLED SDA	GPIO 4	The shared Data line for communication.
Sensor SCL & OLED SCL	GPIO 5	The shared Clock line to keep communication timed correctly.

Code

```
1. #include <Arduino.h>
2. #include <Wire.h>
3. #include <U8g2lib.h>
4. #include <MAX30100.h>
5.
6. // --- Shared I2C Pin Configuration ---
7. #define I2C_SDA 4
8. #define I2C_SCL 5
9.
10. // --- Initialize the I2C OLED Display ---
11. U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);
12. MAX30100 particleSensor;
13.
14. const int SCREEN_WIDTH = 128;
15. int waveBuffer[SCREEN_WIDTH];
16.
17. const int BUFFER_SIZE = 250;
18. uint16_t redBuffer[BUFFER_SIZE];
19. uint16_t irBuffer[BUFFER_SIZE];
20. int bufferIndex = 0;
21.
22. float dcIR = 0;
23. const float beta = 0.95f; // Filter strength for the heartbeat wave
24.
25. float finalBpm = 0.0f;
26. float finalSpO2 = 0.0f;
27. bool fingerPresent = false;
```

```

28.
29. void setup() {
30.     Serial.begin(115200);
31.
32.     // Start the shared I2C bus on our chosen pins
33.     Wire.begin(I2C_SDA, I2C_SCL);
34.     oled.begin();
35.
36.     // Connect to the MAX30100 Sensor
37.     if (!particleSensor.begin()) {
38.         Serial.println("MAX30100 not found. Check wiring or the 1.8V pull-up issue!");
39.         while (1); // Freeze system if sensor is missing
40.     }
41.
42.     // Configure the MAX30100 for SpO2 and HR mode at 100Hz
43.     particleSensor.setMode(MAX30100_MODE_SPO2_HR);
44.     particleSensor.setLedsCurrent(MAX30100_LED_CURR_50MA, MAX30100_LED_CURR_50MA);
45.     particleSensor.setSamplingRate(MAX30100_SAMPRATE_100HZ);
46.
47.     // Fill the screen buffer with a flat line to start
48.     for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
49. }
50.
51. void loop() {
52.     particleSensor.update(); // Tell the library to fetch new data
53.
54.     uint16_t irSample, redSample;
55.
56.     // If there is a new valid reading from the sensor...
57.     if (particleSensor.getRawValues(&irSample, &redSample)) {
58.
59.         // 1. Check if a finger is actually on the sensor
60.         // Note: MAX30100 raw values are lower than the MAX30102
61.         if (irSample < 7000) {
62.             fingerPresent = false;
63.             finalBpm = 0.0f;
64.             finalSpO2 = 0.0f;
65.             bufferIndex = 0;
66.
67.             // Reset wave to flat
68.             for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
69.         } else {
70.             fingerPresent = true;
71.
72.             // 2. Filter the signal to draw the heartbeat wave on the screen
73.             dcIR = (beta * dcIR) + ((1.0f - beta) * irSample);
74.             float acDisplay = irSample - dcIR;
75.
76.             // Shift the old drawing left by 1 pixel to make it scroll
77.             for (int i = 0; i < SCREEN_WIDTH - 1; i++) {
78.                 waveBuffer[i] = waveBuffer[i + 1];
79.             }
80.
81.             // Add the newest pulse reading to the right edge of the screen
82.             int yPixel = 42 - (int)(acDisplay * 0.05f);
83.             waveBuffer[SCREEN_WIDTH - 1] = constrain(yPixel, 18, 54);
84.
85.             // 3. Save the data into our batch array
86.             redBuffer[bufferIndex] = redSample;
87.             irBuffer[bufferIndex] = irSample;
88.             bufferIndex++;
89.
90.             // 4. Once we have 100 readings (1 second of data), do the math
91.             if (bufferIndex >= BUFFER_SIZE) {
92.                 uint16_t minRed = redBuffer[0], maxRed = redBuffer[0];
93.                 uint16_t minIR = irBuffer[0], maxIR = irBuffer[0];
94.                 uint64_t sumRed = 0, sumIR = 0;
95.
96.                 for (int i = 0; i < BUFFER_SIZE; i++) {
97.                     if (redBuffer[i] < minRed) minRed = redBuffer[i];
98.                     if (redBuffer[i] > maxRed) maxRed = redBuffer[i];
99.                     if (irBuffer[i] < minIR) minIR = irBuffer[i];

```

```

100.         if (irBuffer[i] > maxIR) maxIR = irBuffer[i];
101.         sumRed += redBuffer[i];
102.         sumIR += irBuffer[i];
103.     }
104.
105.     float dcR_calc = (float)sumRed / BUFFER_SIZE;
106.     float dcI_calc = (float)sumIR / BUFFER_SIZE;
107.     float acR_calc = (float)(maxRed - minRed);
108.     float acI_calc = (float)(maxIR - minIR);
109.
110.     // 5. Calculate Blood Oxygen (SpO2)
111.     if (acI_calc > 10 && acR_calc > 10) {
112.         float R = (acR_calc / dcR_calc) / (acI_calc / dcI_calc);
113.         float spo2Est = 104.0f - (17.0f * R);
114.         if (spo2Est >= 90.0f && spo2Est <= 100.0f) {
115.             finalSpO2 = spo2Est;
116.         }
117.     }
118.
119.     // 6. Calculate Heart Rate (BPM) by counting the peaks
120.     int lastPeakIdx = -1;
121.     int totalIntervals = 0;
122.     int intervalSum = 0;
123.     float localThresh = dcI_calc;
124.
125.     for (int i = 1; i < BUFFER_SIZE - 1; i++) {
126.         if (irBuffer[i] > localThresh && irBuffer[i-1] <= localThresh) {
127.             if (lastPeakIdx != -1) {
128.                 intervalSum += (i - lastPeakIdx);
129.                 totalIntervals++;
130.             }
131.             lastPeakIdx = i;
132.         }
133.     }
134.
135.     if (totalIntervals > 0) {
136.         float avgIntervalSamples = (float)intervalSum / totalIntervals;
137.         float calculatedBpm = (60.0f * 100.0f) / avgIntervalSamples;
138.         if (calculatedBpm >= 50.0f && calculatedBpm <= 160.0f) {
139.             finalBpm = calculatedBpm;
140.         }
141.     }
142.     bufferIndex = 0; // Reset the batch counter for the next second
143. }
144. }
145. }
146.
147. // 7. Draw the graphics to the OLED Screen
148. oled.clearBuffer();
149.
150. if (fingerPresent && finalBpm > 10.0f) {
151.     // Draw the continuous scrolling waveform
152.     for (int x = 1; x < SCREEN_WIDTH; x++) {
153.         oled.drawLine(x - 1, waveBuffer[x - 1], x, waveBuffer[x]);
154.     }
155.     // Print the live numerical metrics
156.     oled.setFont(u8g2_font_6x10_tf);
157.     oled.setCursor(2, 11); oled.print("BPM: "); oled.print((int)finalBpm);
158.     oled.setCursor(72, 11); oled.print("SpO2: "); oled.print((int)finalSpO2); oled.print("%");
159. } else {
160.     // Prompt the user to place their finger
161.     oled.setFont(u8g2_font_5x7_tf);
162.     oled.setCursor(34, 32); oled.print("PLACE FINGER");
163.     oled.setCursor(36, 44); oled.print("ON THE LENS");
164. }
165.
166. // Draw the top and bottom visual boundary lines
167. oled.drawLine(0, 14, 128);
168. oled.drawLine(0, 56, 128);
169. oled.sendBuffer();
170. }
171.

```

III. Play & Share

- **Implement Signal Quality Index (SQI) Detection:** Modify the block analysis pipeline to compute the variance or standard deviation of peak heights across a window. If the signal is too erratic due to finger movement, display a warning banner like "MOTION ARTIFACT" instead of updating the BPM.
- **Perfusion Index (PI) Calculation:** Calculate the Perfusion Index ($PI = (AC/DC) \times 100\%$). Render a dynamic strength meter on the top OLED bar to guide users on whether their finger pressure is optimal.
- **Pulse Audio Beeper / Haptic Metronome:** Add a piezo buzzer connected to a free GPIO pin. Program the system to sound a short tone on each detected arterial peak, replicating clinical patient monitoring systems.